

1 输入/输出系统 & 管理

1. I/O 系统层次结构：CPU → 通道 → 设备控制器 → 设备

- 在系统重，可能存在多个通道，每个通道可以连接多个控制器，每个控制器可以连接多个物理设备
- 通道负责控制多个设备控制器，减轻 CPU 的 I/O 负担
- 设备控制器负责控制设备并于 CPU 通信
- 设备负责执行 I/O

2. 设备绝对号：系统为每个物理设备分配的唯一编号（物理设备号），用于标识具体的硬件设备

3. I/O 设备按信息交换的单位分类：

- 块设备：信息交换以数据块为单位。基本特征是传输速率较高、可寻址
- 字符设备：信息交换以字符为单位。基本特征是传输速率低、不可寻址，并且通常采用中断 I/O 方式

4. I/O 设备按共享属性分类：

- 独占设备：同一时刻只能由一个进程占用的设备。通常采用静态分配方式
- 共享设备：同一时间段内允许多个进程同时访问的设备。通常采用动态分配方式
 - ① 宏观上同时共享，微观上分时占用
 - ② 通过分时的方式共享使用
 - ③ 必须是可寻址的和可随机访问的设备，从而保证数据的完整性和一致性，提高设备的利用率
- 虚拟设备：通过 SPOOLing 技术将独占设备改造为共享设备，将一个物理设备变为多个逻辑设备

1.1 I/O 接口

1. I/O 接口（I/O 控制器）：主机和外设之间的交界界面，通过接口可以实现主机和外设之间的信息交换
2. 磁盘驱动器由磁头、磁盘和读/写电路组成，故不属于 I/O 接口

1.1.1 I/O 接口的基本结构

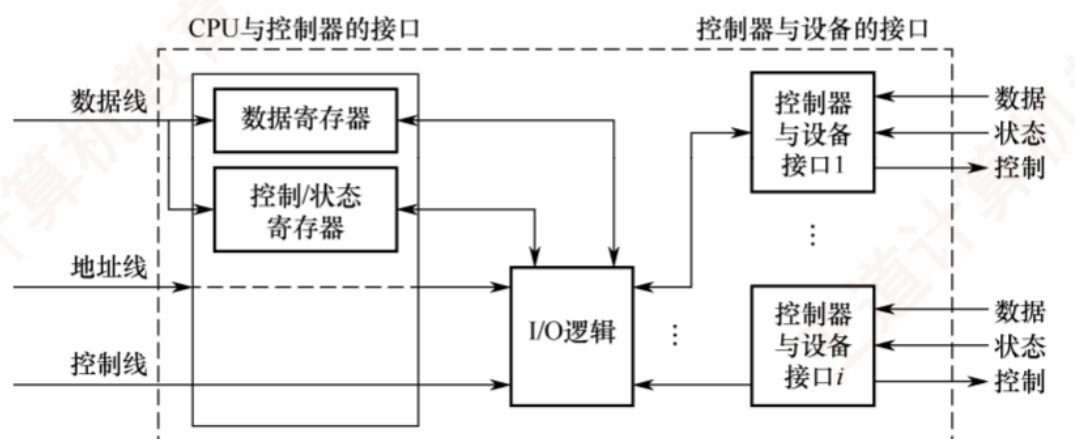


图 5.1 设备控制器的组成

1. 设备控制器与 CPU 的接口：用于实现 CPU 与设备控制器之间的通信
 - 数据线：传送读/写数据、状态信息、控制信息和中断类型号
 - 地址线：传送要访问 I/O 接口中的寄存器的地址
 - 控制线：传送读/写控制信号、中断请求和响应信号、握手信号
2. I/O 逻辑：用于实现对设备的控制
3. 设备控制器与设备的接口
4. I/O 端口：I/O 接口电路中可被 CPU 直接访问的**寄存器**，主要有数据端口（读/写操作）、状态端口（读操作）和控制端口（写操作）
 - 数据缓冲寄存器：暂存与 CPU 或内存之间传送的数据信息
 - 状态寄存器：记录接口和设备的状态信息
 - 控制寄存器：保存 CPU 对外设的控制信息
 - 状态寄存器和控制寄存器在传送方向上是相反的，在访问时间上也是错开的，故合二为一
 - I/O 端口要想被 CPU 访问，就必须要对各个端口进行编址，每个端口对应一个端口地址，编址方式：
 - ① 独立编址（I/O 映射方式）：对所有的 I/O 端口单独进行编址。I/O 端口的地址空间与主存地址空间是两个独立的地址空间。需要设置专门的 I/O 指令来表明访问的是 I/O 地址空间
 - ② 统一编制（存储器映射方式、内存映射 I/O）：把主存地址空间分出一部分给 I/O 端口进行编址，根据地址范围就能区分访问的是 I/O 端口还是主存单元。使用指令系统中的**访存指令**来完成输入/输出操作
5. I/O 指令：对数据缓冲寄存器、状态/控制寄存器的访问操作是通过 I/O 指令完成的，I/O 指令只能在操作系统内核的底层 I/O 软件中使用，是一种特权指令

1.1.2 I/O 接口的功能

1. 识别和接收命令
2. 数据交换
3. 标识和报告设备的状态
4. 地址识别
5. 数据缓冲：接口需设置数据缓冲寄存器，用于数据的暂存，以避免因速度不一致而丢失数据
6. 差错控制

1.2 I/O 控制方式

在输入/输出系统实现主机与 I/O 设备之间的**数据传送**可以采用不同的控制方式：

1. 程序查询方式（程序直接控制方式、程序轮询方式）
2. 程序中断方式（中断驱动方式）
3. DMA 方式（Direct Memory Access, 直接存储器存取）
4. 通道方式：在系统中设有通道控制部件，每个通道都挂接若干外设，主机在执行 I/O 命令时，只需启动有关通道，通道将执行通道程序，从而完成 I/O 操作

1.2.1 程序查询方式（程序直接控制方式、程序轮询方式）

设置一个数据缓冲寄存器和一个设备状态寄存器，主机进行 I/O 操作时，先读取设备的状态并根据设备状态决定下一步操作。根据 CPU 查询设备状态的方式不同，程序查询分为：

1. 独占查询：一旦设备启动，CPU 就一直持续查询接口状态，可能会引发踏步等待现象
 2. 定时查询：CPU 周期性地查询接口状态，每次总是等到条件满足才进行一个数据的传输，传输完成后返回用户程序。需要保证数据不丢失
1. CPU 负责数据的传送，每完成一次数据传送后，将主存地址加 1，计数值减 1
 2. 数据传送单位是字，每次读/写一个字
 3. CPU 和 I/O 设备只能串行工作

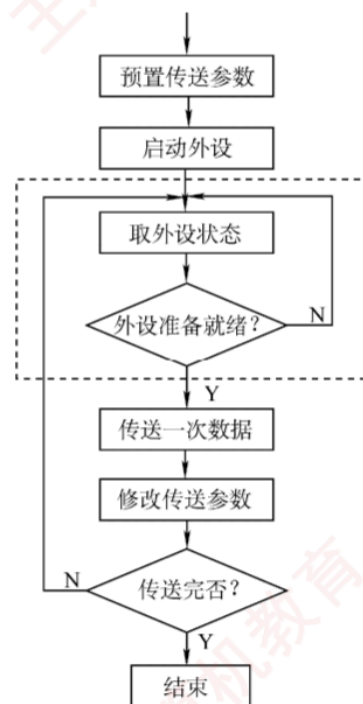


图 7.2 程序查询方式的工作流程

1.2.2 程序中断（中断驱动方式）

1. 程序中断：在计算机执行程序的过程中，出现某些继续处理的异常情况或特殊请求，CPU 暂时中止现程序，而转去对这些异常情况或特殊请求进行处理，处理完毕后再返回到原程序的断点处，继续执行原程序
2. CPU 与外设并行工作，传送与主程序串行工作
3. CPU 会在每个指令周期的末尾检查中断
4. 数据传送单位是字，每次读/写一个字
 - Cache 缺失：Cache 完全是由硬件实现的，不会涉及中断层面
 - 虚拟存储器时效：如缺页 $\Rightarrow$ 缺页中断（内中断）
 - 浮点数运算下溢：当做机器零处理，不会引发中断
 - 浮点数运算上溢：引发浮点溢出异常（内中断）
 - CPU 在执行一条指令的过程中检测异常事件
5. 若准备数据的时间小于中断响应和中断处理的时间，则数据被刷新，造成丢失
6. 中断请求
 - 中断源：请求 CPU 中断的设备或事件
 - 中断请求标记触发器：记录中断时间并区分不同的中断源，其状态为 1 时，表示该中断源有请求
 - 中断请求标记寄存器：由多个中断请求标记触发器组成
7. 中断响应判优：CPU 响应中断请求的先后顺序，通常通过硬件排队器或终端查询程序实现
 - 中断优先级包括：

① 响应优先级：由硬件线路或查询程序的查询顺序决定，不可动态改变

② 处理优先级：可利用中断屏蔽技术动态调整，以实现多重中断

- 可屏蔽中断：通过 INTR 线发出，优先级最低，在关中断模式下不被响应
- 不可屏蔽中断：通过 NMI 线发出，用于处理紧急和重要的事件，优先级最高
- 不可屏蔽中断 > 可屏蔽中断
- 高速设备 > 低速设备
- 输入设备 > 输出设备
- 实时设备 > 普通设备

8. CPU 响应中断的条件

- 中断源有中断请求
- CPU 允许中断及开中断（异常和不可屏蔽中断不受此限制）
- 一条指令执行完毕（异常不受此限制），且没有更紧迫的任务

9. 中断隐指令：CPU 响应中断后，经过某些操作，转去执行中断服务程序，这些操作是有硬件直接实现的，即中断隐指令，其本质是CPU 内部硬件控制逻辑自动完成的一系列操作，所完成的操作：

- 关中断：CPU 响应中断后，首先要保护程序的断点和现场信息，在保护断点和现场的过程中，CPU 不能响应更高级中断源的中断请求
- 保存断点：将原程序的断点保存在栈或特定寄存器中
- 引出中断程序：识别中断源，将对应的服务程序入口地址送入程序计数器 PC。识别中断源方法

① 硬件向量法/向量中断

② 软件查询法/非向量中断

- 中断向量：每个中断源都有一个唯一的类型号，每个中断类型号都对应一个中断服务程序，每个中断服务程序都有一个入口地址，即中断向量
- 中断向量地址：中断向量的地址，也是中断服务程序入口地址的地址
- 中断向量表：系统中的全部中断向量
- 中断向量法：CPU 响应中断后，通过识别中断源获得中断类型号，然后据此计算出对应中断向量的地址，再根据地址从中断向量表中取出中断服务程序的入口地址，并送入程序计数器 PC，以转去执行中断服务程序
- 向量中断：采用中断向量法的中断

10. 中断处理过程

- 前三步由中断隐指令（硬件自动）完成
- 第四步之后都是由中断服务程序完成
- 第五步开中断：允许更高级中断请求得到响应，已实现中断嵌套
- 最后一步：中断返回指令，需要修改 PC 值，并且要将 CPU 中的所有寄存器都恢复到中断前的状态

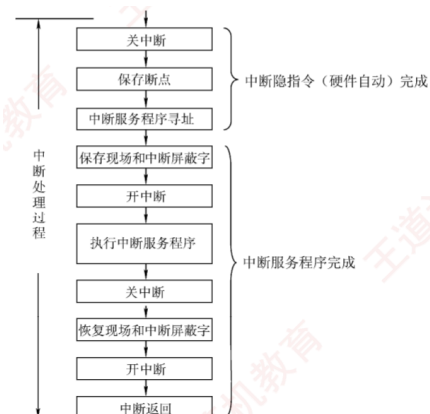


图 7.4 一个支持中断嵌套的典型处理流程

11. 单重中断：在 CPU 执行中断服务程序的过程中，若又出现了新的优先级更高的中断请求，而 CPU 对新的中断请求不予响应
12. 多重中断（中断嵌套）：若 CPU 暂停现行的中断服务程序，转去处理新的中断请求
13. **中断处理优先级**：多重中断的实际优先级处理次序，可以利用中断屏蔽技术动态调整。通过在中断系统中设置**中断屏蔽字**寄存器实现。每个中断源对应中断屏蔽字寄存器中的某一位
 - 1：表示屏蔽该中断源的请求
 - 0：表示可以正常请求
 - 中断屏蔽字：中断屏蔽字寄存器的内容
 - 中断屏蔽字越多 1，优先级越高
 - 每个屏蔽字至少有一个 1 $\Rightarrow$ 屏蔽自身中断
 - 所有中断请求信号都会与中断屏蔽字寄存器中对应位的非值进行逻辑与，然后被送入中断判优电路
 - 通过配置中断屏蔽字，可以动态改变中断处理优先级

1.2.3 DMA 方式 (Direct Memory Access, 直接存储器存取)

1. DMA 方式（直接存储器存取方式）：完全由硬件进行成组信息传送的控制方式，具有程序中中断方式的优点（即在数据准备阶段，CPU 与外设并行工作），外设和内存之间开辟了一条直接数据通路，传送信息不再经过 CPU
2. 特点
 - 基本传送单位是数据块，而不再是字
 - 所传送的数据，是从设备直接送入内存的，或者相反，而不再经过 CPU
 - 仅在传送一个或多个数据块的开始和结束时，才需要 CPU 干预
3. 只有具有 DMA 接口的设备才能产生 DMA 请求
4. **DMA 优先级高于中断请求**
5. ★★CPU 会在每个机器周期（总线事务、存取周期、总线周期、当前流水段长度）结束后检查是否有 DMA 请求以及响应
6. I/O 与主机并行工作，程序与传送并行工作
7. DMA 控制器（DMA 接口）：对数据传送过程中进行控制的硬件
 - 当 I/O 设备需要进行数据传送时，**（外部设备）向 DMA 控制器发出 DMA 请求信号**，通过 DMA 控制器向 CPU 提出 DMA 传送请求，CPU 响应之后将让出系统总线，有 DMA 控制器接管总线进行数据传送 $\Rightarrow$ DMA 控制器具有控制相同总线的能力

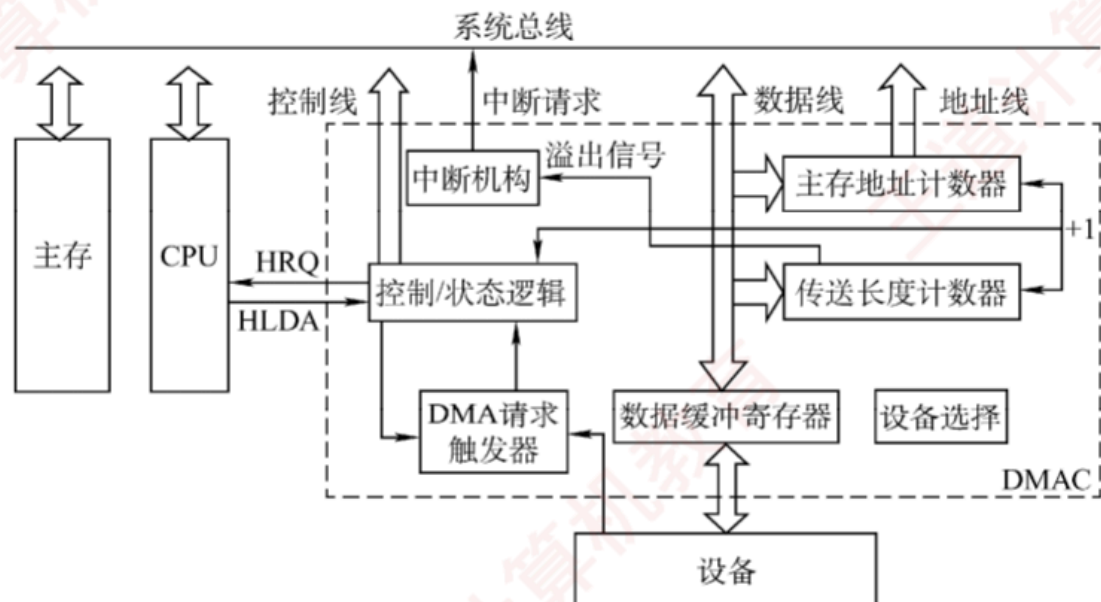


图 7.8 一个简单的 DMA 控制器

• 组成

- ① CR (Command Register, 命令/状态寄存器): 用于存放 CPU 发来的 I/O 命令或设备的状态信息
- ② MAR (Memory Address Register, 主存地址计数器、内存地址寄存器): 存放交换数据的主存地址
- ③ DR (Data Register, 数据寄存器、数据缓冲寄存器): 暂存每次传送的数据。**DMA 接口与主存之间传送单位为字**
- ④ DC (Data Counter, 传送长度计数器、数据计数器): 记录传送数据的长度
- ⑤ DMA 请求触发器
- ⑥ 控制/状态逻辑: 用于指定传送方向, 修改传送参数, 并对 DMA 请求信号、CPU 响应信号进行协调和同步
- ⑦ 中断机构: 当一批数据传送完毕后触发中断机构, 向 CPU 提出中断请求

8. DMA 的传送方式: 当 I/O 设备和 CPU 同时访问主存时, 可能发生冲突, 避免冲突方法

- 停止 CPU 访存
- 周期挪用: 由于 I/O 访存的有优先级高于 CPU 访存 (I/O 不立即访存则可能丢失数据), I/O 设备挪用一個存取周期, 传送完一个**数据字**后立即释放总线。**单字传送方式**。I/O 设备有 DMA 请求时, 有以下情况
 - ① CPU 不在访存
 - ② CPU 正在访存: 等待存取周期结束, CPU 再将总线占有权让出
 - ③ 同时请求访存: CPU 要暂时放弃总线占有权
- 交替访存: 将 CPU 的工作周期分成两个时间片, 一个给 CPU 访存, 另一个给 DMA 访存。适用于 CPU 的工作周期比主存存取周期长的情况

9. DMA 传送过程

- 预处理: 由 CPU 完成一些必要的准备工作。运行在内核态。由请求 I/O 的进程负责, 故请求 I/O 的进程处于运行态

- 初始化 DMA 控制器中的有关寄存器、设置传送方向、测试并启动设备等
- CPU 继续执行原程序，直到 I/O 设备准备好发送的数据（输入情况）或接受的数据（输出情况）时，I/O 设备向 DMA 控制器发送 DMA 请求，再由 DMA 控制器向 CPU 发出总线请求
- 数据传送：DMA（硬件）完成，DMA 以数据块为基本传送单位（CPU 一次交给 DMA 的任务是多少）
- 后处理：DMA 控制器向 CPU 发出中断请求，CPU 执行中断服务程序做 DMA 结束处理。运行在内核态。此时请求 I/O 的进程处于阻塞态

1.2.4 计算

1. 由于端口有限，故必须在外设传输完端口大小的数据时访问端口，以防止数据未被及时读取而丢失
2. 外设准备数据时间 = $\frac{\text{外设准备的数据量}}{\text{数据率}}$
3. CPU 用于外设 I/O 的时间占 CPU 总时间的百分比 = $\frac{\text{每秒 CPU 用于外设 I/O 的时钟周期}}{\text{主频}}$
4. 外设每秒 DMA 次数 = $\frac{\text{数据率}}{\text{每次 DMA 传送块大小}}$

1.3 I/O 软件层次结构

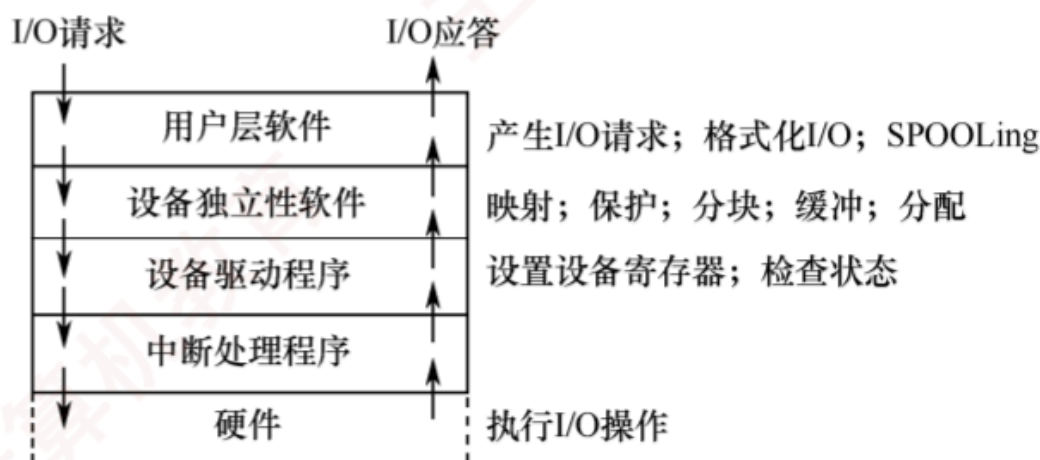


图 5.5 I/O 层次结构

TODO: 区分每层的范围???

1. 用户层软件
2. 设备独立性软件（设备无关性）：实现用户程序与设备驱动器的统一接口、设备命名、设备保护，以及设备的分配与释放等，同时为设备管理和数据传送提供必要的存储空间
 - 让用户程序不用关心具体设备，程序只面对逻辑设备，设备独立性软件负责逻辑设备与物理设备的映射
 - 主要功能：
 - ① 执行所有设备的公有操作

② 向用户层（或文件层）提供统一接口

3. 设备驱动程序：与硬件直接相关，每类设备需要配置一个设备驱动程序

- 负责具体实现系统对设备发出的操作指令，驱动 I/O 设备工作的驱动程序，是 I/O 进程和设备控制器之间的通信程序
- 通常以进程的形式存在

4. 中断处理程序：和硬件交互

1.3.1 逻辑设备名 VS 物理设备名

1. 应用程序所用的设备不局限于某个具体的物理设备

2. 在应用程序中，使用逻辑设备名来请求使用某类设备，而在系统实际执行时，必须将逻辑设备名映射成物理设备名

3. 逻辑设备名好处：

- 增加设备分配的灵活性
- 易于实现 I/O 重定向（I/O 重定向：用于 I/O 操作的设备可以更换，而不必改变应用程序）

4. 逻辑设备表（Logical Unit Table, LUT）：逻辑设备名与物理设备名的映射

- LUT = 逻辑设备名 + 物理设备名 + 设备驱动程序的入口地址
- 两种方式设置 LUT：

① 整个系统重只设置一张 LUT

② 为每个用户设置一张 LUT

1.3.2 设备分配

1. 数据结构

- 设备控制表（DCT）：每个设备对应一张 DCT，表中的表项就是设备的各个属性
- 控制器控制表（COCT）：每个设备控制器都对应一张 COCT，操作系统根据 COCT 的信息对控制器进行操作和管理
- 通道控制表（CHCT）：每个通道对应一张 CHCT，操作系统根据 CHCT 的信息对通道进行操作和管理
- 系统设备表（SDT）：整个系统只有一张 SDT，记录已连接到系统重的所有物理设备的情况，每个物理设备对应一个表目

2. 设备分配时应考虑的因素

- 设备的固有属性：独占设备 Or 共享设备 Or 虚拟设备
- 设备的分配算法：FCFS 算法 Or 最高优先级优先算法
- 设备分配中的安全性
 - ① 安全分配方式：每当进程发出 I/O 请求后，便进入阻塞态，知道其 I/O 操作完成时才被唤醒
 - ② 不安全分配方式：进程在发出 I/O 请求后仍继续运行，需要是会继续发出 I/O 请求，仅当进程所请求的设备已被另一进程占用时，才进入阻塞态

3. 步骤

- 分配设备
- 分配控制器
- 分配通道
- 只有设备、控制器和通道都分配成功，设备分配才算成功，之后便可启动设备进行数据传送

1.4 应用程序 I/O 接口

1. 应用程序 I/O 接口是软件和操作系统之间的接口，I/O 接口（I/O 控制器）是 CPU/内存和外部设备之间的硬件接口
2. 分类：
 - 字符设备接口
 - 字符设备：数据的存取和传输是以字符为单位的设备
 - 基本特征：传输速率较低、不可寻址，并且在输入/输出是通常采用中断驱动方式
 - 块设备接口
 - ① 块设备：数据的存取和传输是以数据块为单位的设备
 - ② 基本特征：传输速率较高，可寻址。磁盘设备的 I/O 通常采用 DMA 方式
 - 网络设备接口：网络 I/O 接口为网络套接字接口
3. 模式
 - 阻塞 I/O：当用户进程调用 I/O 操作时，进程就被阻塞，并移到阻塞队列，I/O 操作完成后，进程才被唤醒，移到就绪队列
 - 非阻塞 I/O：当用户进程调用 I/O 操作是，不阻塞该进程，但进程需要不断询问 I/O 操作是否完成，在 I/O 执行阶段，进程还可以做其他事情

1.5 缓冲管理

1. 磁盘高速缓存技术：利用内存中的存储空间来暂存从磁盘中读出的一系列盘块中的信息。磁盘高速缓存逻辑上属于磁盘，物理上是驻留在内存中的盘块

1.5.1 缓冲区（Buffer）

1. 缓冲区是一种临界资源，在使用缓冲区时是互斥访问的
2. 引入缓冲区的目的
 - 缓和 CPU 与 I/O 设备间速度不匹配的矛盾
 - 减少对 CPU 的中断频率，放宽对 CPU 中断响应时间的限制
 - 解决基本数据单元大小不匹配的问题
 - 提高 CPU 和 I/O 设备之间的并行性
3. 缓冲区的实现方法
 - 采用硬件缓冲器
 - 利用内存作为缓冲区
4. 根据系统设置缓冲区的个数，缓冲技术分类：

- T: 从设备将一块数据输入缓冲区的时间
- M: 操作系统将该缓冲区中的数据传送到工作区的时间
- C: CPU 对这一块数据进行处理的时间
- 单缓冲: 每当用户进程发出一个 I/O 请求, 操作系统便在内存中为之分配一个缓冲区
 - ① 一个缓冲区的大小就是一个块
 - ② 必须等缓冲区装满后才能从缓冲区取走数据
 - ③ T 和 C 是可以并行的
 - ④ 单缓冲区处理每块数据的平均时间: $\text{Max}(C, T) + M$
 - ⑤ 缓冲区是共享资源, 故使用时必须互斥
- 双缓冲 (缓冲对换)
 - ① C 和 M 是可以与 T 并行的
 - ② 双缓冲区处理每块数据的平均时间: $\text{Max}(C + M, T)$
 - ③ 可以实现双向数据传输
 - ④ 若输入和输出的速度相差较大时, 效果不理想
- 循环缓冲 (多缓冲机制): 包含多个大小相等的缓冲区, 多个缓冲区链接称一个循环队列
- 缓冲池: 包含一个用于管理自身的数据结构和一组操作函数的管理机制, 用于管理多个缓冲区
 - ① 缓冲区按其使用状况分为
 - (a) 空缓冲队列
 - (b) 输入队列
 - (c) 输出队列
 - ② 4 中工作缓冲区
 - (a) hin: 用于收容输入数据的工作缓冲区
 - (b) sin: 用于提取输入数据的工作缓冲区
 - (c) hout: 用于收容输出数据的工作缓冲区
 - (d) sout: 用于提取输出数据的工作缓冲区
 - ③ 4 种工作方式
 - (a) 收容输入
 - (b) 提取输入
 - (c) 收容输出
 - (d) 提取输出

1.5.2 SPOOLing 技术（假脱机技术）

1. 目的：缓和 CPU 的高速型与 I/O 设备的低速性之间的矛盾，将独占设备改造成共享设备的技术
2. 多道程序技术（Multiprogramming）：让 CPU 和 I/O 设备尽量同时工作，提高系统资源利用率
3. 多道程序技术是 SPOOLing 技术的基础。SPOOLing 利用多道程序技术，使输入进程、输出进程和用户进程并发执行，并借助磁盘作为缓冲区，将独占设备虚拟为共享设备，从而提高 CPU 和设备的利用率
4. SPOOLing 技术是一种以空间换时间的技术
5. SPOOLing 技术：利用专门的外围控制机，先将低速 I/O 设备上的数据传送到高速磁盘上，或者相反，当 CPU 需要输入数据时，便可直接从磁盘中读取数据；反之，当 CPU 需要输出数据时，也能以很快的速度将数据先输出到磁盘上
6. 例子：扫描仪、打印机
7. 组成
 - 输入井和输出井
 - 输入缓冲区和输出缓冲区
 - 输入进程和输出进程
8. 核心程序
 - 预输入程序
 - 缓输出程序
 - 井管理程序